

Neural Networks Training and Testing (Hyperparameters)

Justine Bradley Tulio
Bachelor of Science in Information
Technology
Jose Rizal University
Manila, Philippines
justinebradley.tulio@my.jru.edu

Abstract—This study presents a series of neural network experiments for classifying customer feedback into Positive Praise, Negative Complaint, and Urgent Support categories. The experiments began by comparing a TensorFlow/Keras neural network with the previous PT-P1 no-code approach. Initial experiments examined different learning rates, hidden-layer dimensions, and dropout rates to observe how these settings affected model learning and generalization. The results showed cases of both underfitting and overfitting, particularly when the model capacity and learning rate were not well matched with the limited dataset. The training dataset was later expanded from 30 to 900 balanced samples, with 300 instances for each category. Custom experiments then explored additional optimization, architecture, and text-processing adjustments. Early Stopping was also applied to monitor validation loss and restore the weights from the best epoch. The custom experiments achieved validation accuracies between 98.52% and 99.26%, showing that data expansion combined with controlled optimization substantially improved model performance.

Keywords—*Neural Network, TensorFlow, Keras, hyperparameter optimization, loss curve, Early Stopping, text classification*

I. INTRODUCTION AND PT-P1 BASELINE PERFORMANCE

This exercise focused on understanding neural network training and optimization by moving from the automated no-code approach used in PT-P1 to a configurable TensorFlow/Keras implementation. The no-code approach made it easier to train and test a classification model because much of the learning process was handled automatically. In comparison, the Keras implementation provided direct control over parameters such as the learning rate, hidden-layer dimension, dropout rate, batch size, number of epochs, and optimizer.

The original PT-P1 dataset contained 30 customer-feedback samples divided equally among three categories: Positive Praise, Negative Complaint, and Urgent Support. In the previous no-code activity, the model was evaluated using accuracy, precision, recall, and F1-score. These results served as a reference for observing how a neural network behaved when its architecture and training settings could be adjusted directly.

The Keras model used an Embedding layer, GlobalAveragePooling1D, a Dense hidden layer, Dropout, and a Softmax output layer. The dataset was divided into 70% training, 15% validation, and 15% testing. During training, the experiments monitored training loss, validation loss, training accuracy, validation accuracy, weighted precision, weighted recall, and weighted F1-score.

One important observation from the initial experiments was that having more control did not automatically result in better performance. Some configurations caused the model to learn too slowly, while others allowed it to fit the training data too strongly. Because of this, the loss curves became useful for understanding what was happening during training instead of relying only on the final accuracy.

II. EXPERIMENTAL METHODOLOGY AND HYPERPARAMETERS CONFIGURATIONS

Before training, the customer-feedback text was converted into numerical values that could be processed by the neural network. A vocabulary was created using `build_vocab()`, with index 0 reserved for padding and index 1 for unknown words. Each feedback entry was then converted into a sequence of vocabulary IDs using `text_to_sequence()`. Padding or truncation was applied according to the configured `MAX_LEN` so that all inputs had a consistent length.

The baseline architecture consisted of an Embedding layer followed by GlobalAveragePooling1D, a Dense hidden layer using ReLU activation, a Dropout layer, and a final Dense output layer using Softmax activation. Sparse Categorical Cross-Entropy was used as the loss function. Adam and SGD were available as optimizer choices depending on the experimental configuration.

The initial experiments modified selected hyperparameters to observe different model behaviors.

TABLE I. INITIAL KERAS EXPERIMENT CONFIGURATIONS

Experiment	Learning Rate	Hidden Dim	Dropout	Epochs
Exp-01	0.005	64	0.1	15
Exp-02	0.0001	16	0.0	15
Exp-03	0.05	256	0.0	15
Exp-04	0.005	128	0.4	15

EXP-01 represented the initial Keras baseline configuration. EXP-02 reduced the learning rate and hidden-layer dimension to observe what would happen when the model learned more slowly with lower capacity. EXP-03 moved in the opposite direction by using a much higher learning rate, a larger hidden dimension, and no Dropout. EXP-04 then introduced stronger regularization using a dropout rate of 0.4.

After completing these initial experiments, custom experiments were performed. The most important change was introduced in EXP-05A, where the dataset was expanded from 30 to **900 balanced samples**, consisting of 300 samples for each classification category. The succeeding custom experiments continued using the expanded dataset while examining different optimization, architectural, and text-processing configurations.

TABLE II. CONFIGURATIONS CUSTOM OPTIMIZATION CONFIGURATIONS

Experiment	Learning Rate	Hidden Dim	Dropout	Epochs
Exp-05A	0.005	128	0.4	30
Exp-05B	0.001	128	0.4	30
Exp-05C	0.001	256	0.4	30
Exp-05D	0.001	256	0.4	50
Exp-05E	0.001	256	0.4	50
Exp-05F	0.001	256	0.4	50

Table II presents the main hyperparameter configurations used for the custom experiments. EXP-05A established the first custom configuration using the expanded dataset. The succeeding experiments explored further changes in optimization, architecture, and text preprocessing. Some experiments also involved settings not shown in Table II, such as batch size, sequence length (MAX_LEN), vocabulary size (MAX_VOCAB_SIZE), or additional architectural adjustments. These experiments were performed to observe whether combining different optimization strategies could further improve or maintain model generalization.

Early Stopping was also introduced during the custom experiments. The callback monitored val_loss using a patience value of three and restore_best_weights=True. This meant that if validation loss stopped improving, the model was allowed three additional epochs before training stopped. The weights from the best-performing epoch were then

restored. Therefore, the epoch where training stopped was not necessarily the same as the epoch containing the best model.

III. PERFORMANCE MONITORING AND LOSS CURVE ANALYSIS

TABLE III. LOSS AND ACCURACY RESULTS

Experiment	Train Loss	Val. Loss	Train Acc.	Val Acc.
Exp-01	0.6487	1.2757	100%	50.00%
Exp-02	1.0922	1.0983	47.62%	50.00%
Exp-03	0.0000	11.8350	100%	50.00%
Exp-04	0.5849	1.0705	100%	50.00%
Exp-05A	0.0258	0.0254	99.37%	99.26%
Exp-05B	0.0049	0.0327	100%	98.52%
Exp-05C	0.0048	0.0303	100%	99.26%
Exp-05D	0.0049	0.0321	100%	99.26%
Exp-05E	0.0045	0.0266	100%	99.26%
Exp-05F	0.0025	0.0327	100%	98.52%

TABLE IV. RESULTS VALIDATION CLASSIFICATION METRICS

Experiment	Precision	Recall	F1-Score.
Exp-01	0.6250	0.5000	0.4583
Exp-02	0.3333	0.5000	0.4000
Exp-03	0.6250	0.5000	0.5000
Exp-04	0.5833	0.5000	0.4583
Exp-05A	0.9927	0.9926	0.9926
Exp-05B	0.9855	0.9852	0.9851
Exp-05C	0.9927	0.9926	0.9926
Exp-05D	0.9927	0.9926	0.9926
Exp-05E	0.9927	0.9926	0.9926
Exp-05F	0.9857	0.9852	0.9852

A. Abbreviations and Acronyms EXP-01 Keras Baseline

EXP-01 produced a final training loss of **0.6487** and reached 100% training accuracy. However, validation loss reached **1.2757**, while validation accuracy remained at only **50.00%**. Precision was **0.6250**, recall was **0.5000**, and the weighted F1-score was **0.4583**.

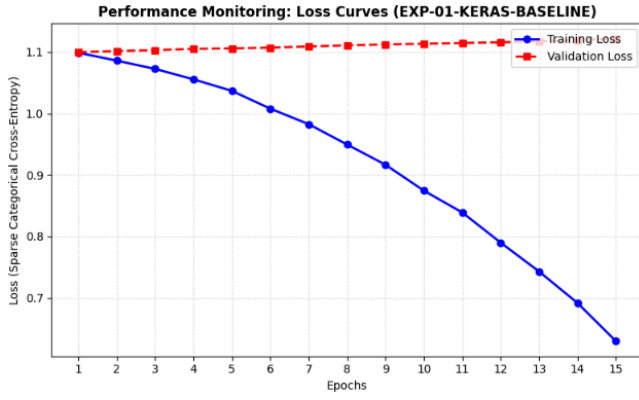


Fig. 1. Training and validation loss curves for EXP-01.

The loss curve shows training loss continuously decreasing while validation loss remains high and gradually moves in the opposite direction. This separation indicates **overfitting**. The model was becoming increasingly accurate on the training examples, but the same improvement was not being transferred to unseen validation samples.

B. EXP-02: Reduced Learning Rate and Model Capacity

EXP-02 reduced the learning rate to 0.0001 and the hidden dimension to 16. Training loss was **1.0922**, while validation loss was **1.0983**. Training accuracy reached only **47.62%**, and validation accuracy was **50.00%**. The experiment also produced a precision of **0.3333**, recall of **0.5000**, and F1-score of **0.4000**.

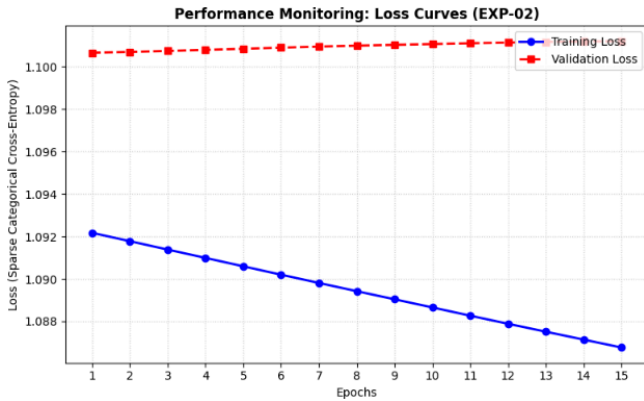


Fig. 2. Training and validation loss curves for EXP-02

The two loss curves remained close together, but both stayed relatively high and changed very little. This indicates **underfitting** rather than good generalization. The model was not learning the training data sufficiently. The very small learning rate combined with the smaller hidden layer limited how much the model could learn during the 15 epochs.

C. EXP-03: Increased Capacity and Learning Rate

EXP-03 used a hidden dimension of 256, a learning rate of 0.05, and no Dropout. The experiment reached **100% training accuracy** with a recorded training loss of **0.0000**. In contrast, validation accuracy remained at **50.00%**, while validation loss increased dramatically to **11.8350**.

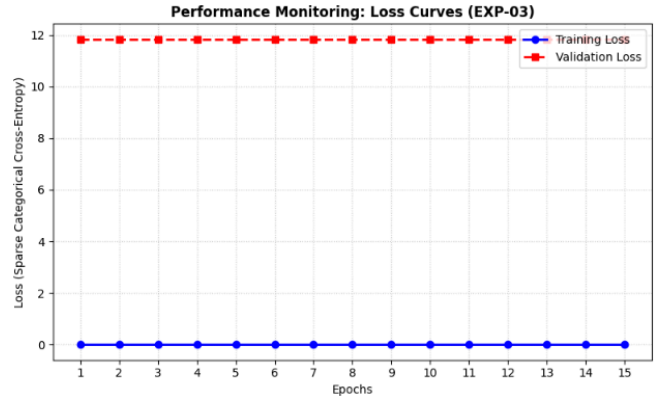


Fig. 3. Training and validation loss curves for EXP-03.

This was the clearest example of **severe overfitting** among the initial experiments. The model almost completely fit the training data, but this did not translate into better validation performance. The combination of a larger hidden layer, high learning rate, no Dropout, and a very small dataset created a condition where the model could fit the available examples too aggressively.

D. EXP-04: Stronger Dropout Regularization

EXP-04 used a hidden dimension of 128 with a dropout rate of 0.4. Training loss reached **0.5849**, while validation loss was **1.0705**. Training accuracy reached 100%, but validation accuracy remained at **50.00%**. The weighted F1-score was **0.4583**.

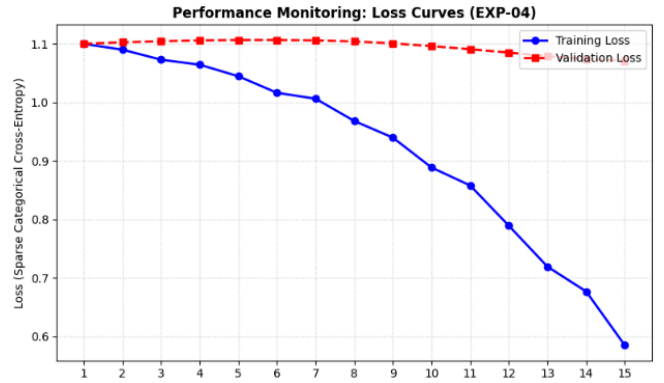


Fig. 4. Training and validation loss curves for EXP-04.

The model continued to show **overfitting**, although the behavior was less extreme than EXP-03. Increasing Dropout helped reduce the model's dependence on individual training features, but it was not enough to overcome the limitations of the small dataset. This experiment helped show that regularization can control model capacity, but it cannot replace sufficient and representative training data.

E. EXP-05A: Dataset Expansion

EXP-05A introduced the largest change in the experiments by increasing the dataset to **900 balanced samples**, with 300 examples for Positive Praise, 300 for Negative Complaint, and 300 for Urgent Support.

The experiment achieved a training loss of **0.0258** and validation loss of **0.0254**. Training accuracy reached **99.37%**, while validation accuracy reached **99.26%**. Precision was **0.9927**, recall was **0.9926**, and the F1-score was **0.9926**.

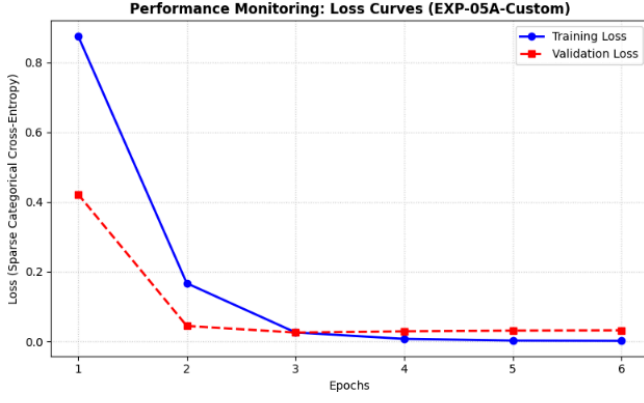


Fig. 5. Training and validation loss curves for EXP-05A.

Compared with EXP-01 through EXP-04, the difference was substantial. Training and validation performance remained very close, with only a **0.11 percentage-point difference** between their accuracies. The training and validation losses were also nearly identical.

Based on the recorded metrics, EXP-05A demonstrated the clearest **good-fit behavior** among the experiments. The model learned the training data effectively while maintaining almost the same performance on the validation data.

Early Stopping was also applied during this experiment. Training stopped after validation loss failed to improve for the specified patience period, and the model weights from the best epoch were restored.

F. EXP-05B: Learning Rate Adjustment

EXP-05B reduced the learning rate to 0.001. It achieved a training loss of **0.0049** and validation loss of **0.0327**. Training accuracy reached 100%, while validation accuracy reached **98.52%**. Precision was **0.9855**, recall was **0.9852**, and F1-score was **0.9851**.

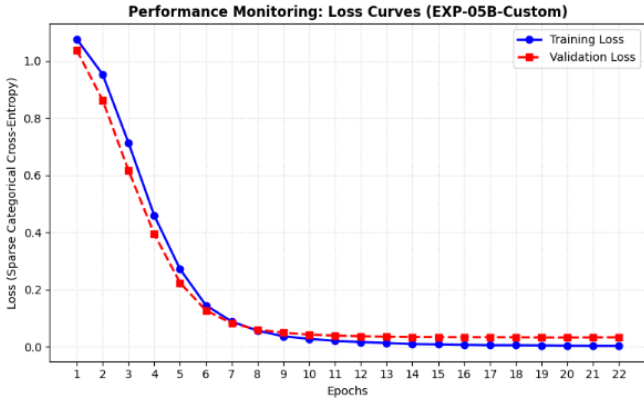


Fig. 6. Training and validation loss curves for EXP-05B.

The lower learning rate resulted in a more gradual decline in loss. Training continued until Early Stopping was triggered, after which the weights from the best epoch were restored. The small separation between training and validation loss toward the later epochs suggests a slight tendency toward overfitting, but validation performance remained very high.

G. EXP-05C: Increased Hidden Dimension

EXP-05C increased the hidden dimension from 128 to **256** while retaining the expanded dataset. Training loss reached **0.0048**, while validation loss was **0.0303**. Training accuracy reached 100%, and validation accuracy reached **99.26%**. Precision was **0.9927**, recall was **0.9926**, and F1-score was **0.9926**.

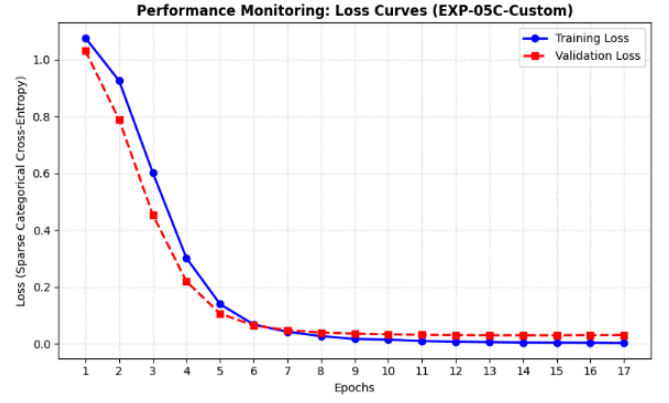


Fig. 7. Training and validation loss curves for EXP-05C.

This experiment provided an interesting comparison with EXP-03. Both experiments used a larger hidden dimension, but EXP-03 severely overfit the smaller dataset while EXP-05C performed strongly using the expanded dataset. This suggests that greater architectural capacity can become useful when enough training data is available to support it.

H. EXP-05D: Text Preprocessing Adjustment

EXP-05D focused on text preprocessing by adjusting the sequence-length configuration while maintaining the larger hidden dimension. It produced a training loss of **0.0049** and validation loss of **0.0321**. Training accuracy was 100%, while validation accuracy reached **99.26%**. Precision was **0.9927**, recall was **0.9926**, and F1-score was **0.9926**.

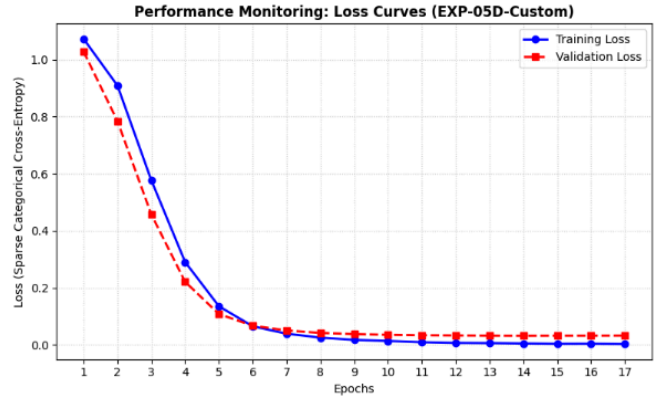


Fig. 8. Training and validation loss curves for EXP-05D.

The result was very similar to EXP-05C. This suggests that adjusting the sequence length did not provide a large additional improvement for the current dataset. One possible explanation is that the previous sequence length was already sufficient to capture most of the useful information contained in the customer-feedback samples.

I. EXP-05E: Combined Architecture and Optimization

EXP-05E combined architecture and optimization adjustments. Training loss reached **0.0045**, while validation loss reached **0.0266**. Training accuracy was 100%, and validation accuracy was **99.26%**. Precision reached **0.9927**, recall reached **0.9926**, and the weighted F1-score was **0.9926**.

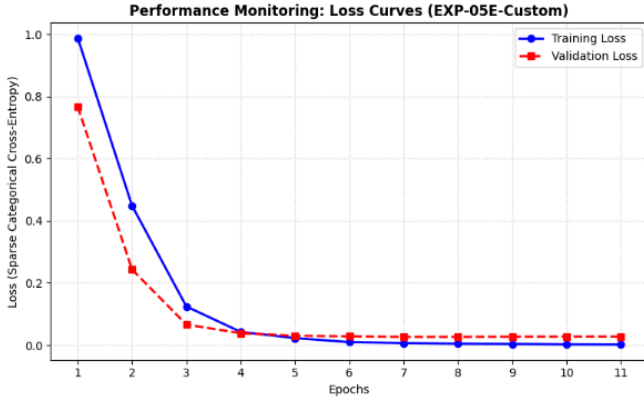


Fig. 9. Training and validation loss curves for EXP-05E.

Among EXP-05B through EXP-05F, EXP-05E recorded the **lowest validation loss at 0.0266** while maintaining the higher validation accuracy of 99.26%. The loss curve shows that the model learned quickly during the earlier epochs before validation performance began to stabilize.

This result demonstrates that combining architectural and optimization changes can maintain strong generalization. However, its overall classification performance remained similar to EXP-05A, EXP-05C, and EXP-05D, which suggests that the expanded dataset had already allowed the model to reach a high level of performance.

J. EXP-05C: Increased Hidden Dimension

EXP-05F combined architecture and text-processing adjustments. It achieved a training loss of **0.0025**, which was the lowest recorded training loss among the custom experiments. Validation loss, however, was **0.0327**. Training accuracy reached 100%, while validation accuracy reached **98.52%**. Precision was **0.9857**, recall was **0.9852**, and F1-score was **0.9852**.

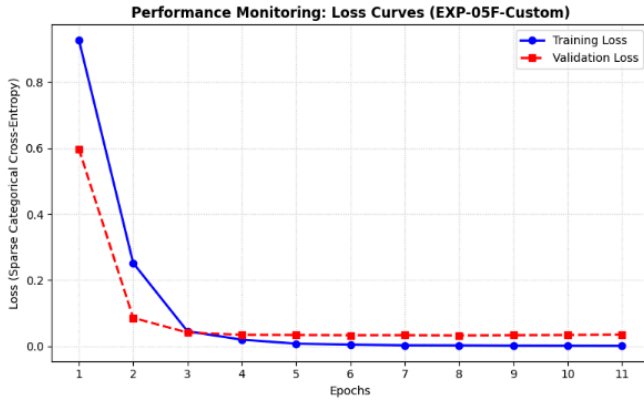


Fig. 10. Training and validation loss curves for EXP-05F.

Although EXP-05F achieved the lowest training loss, it did not produce the highest validation performance. This was an important observation because it showed that making the model fit the training data more closely does not necessarily improve its ability to generalize.

K. Overall Experimental Comparison

Looking across all ten experiments, there is a clear difference between the initial experiments and the custom experiments using the expanded dataset.

EXP-01, EXP-03, and EXP-04 reached 100% training accuracy but only 50.00% validation accuracy. These results showed that strong training performance alone was not enough to conclude that the model had learned useful general patterns. EXP-02 demonstrated the opposite problem, where the model did not sufficiently learn even the training data.

The largest improvement occurred between **EXP-04 and EXP-05A**. Validation accuracy increased from **50.00% to 99.26%**, while weighted F1-score increased from **0.4583 to 0.9926**. This was a much larger improvement than what was achieved by adjusting learning rate, hidden dimension, or Dropout alone in the initial experiments.

Among the custom experiments, EXP-05A showed the closest balance between training and validation performance. EXP-05E also performed strongly and recorded the lowest validation loss among EXP-05B through EXP-05F.

At the same time, EXP-05F showed why training loss should not be used by itself to determine the best model. Although it achieved the lowest training loss at **0.0025**, its validation accuracy of **98.52%** was slightly lower than the 99.26% achieved by EXP-05A, EXP-05C, EXP-05D, and EXP-05E.

IV. DISCUSSION AND DEBRIEF

A. Code-Based Optimization vs. No-Code Heuristics

The no-code approach in PT-P1 was easier because most of the training process was handled automatically. In comparison, the Keras experiments gave more control over how the model was trained and allowed different settings to be tested directly. This made the process more challenging, but it also made it easier to understand why the model was underfitting, overfitting, or improving. Overall, no-code tools offer simplicity and faster experimentation, while code-based training provides greater control and a clearer understanding of model behavior.

B. Diagnosing Model Fit Through Loss Curves and Regularization

The loss curves helped identify the different model-fit conditions. EXP-02 showed underfitting because both training and validation performance remained weak. EXP-03 showed the clearest overfitting because training performance became nearly perfect while validation performance remained poor. Stronger regularization in EXP-04 reduced the severity of overfitting, but the model still had difficulty generalizing. This showed that adjusting model capacity and regularization can improve training behavior, but these settings also need to match the available data.

C. Data Volume and Diversity vs. Architectural Capacity

Increasing the dataset from 30 to 900 balanced samples produced the largest improvement in the experiments. Starting with EXP-05A, validation accuracy increased to approximately 99%, and the training and validation results became much closer. The later experiments showed that increasing model capacity further did not always improve performance. This suggests that a neural network needs enough varied and representative data to support its capacity. A larger model with too little data can memorize the training examples instead of learning patterns that generalize well.

D. Neural Network Training Pipeline

The training process followed the main neural network pipeline discussed in the lecture. The customer feedback was first converted into numerical representations and useful features were automatically extracted before classification. During the forward pass, these features moved through the network to produce a prediction. The prediction was then compared with the correct category to calculate the loss or error. During the backward pass, the model determined how much each learned parameter contributed to that error and adjusted its weights to improve

the next prediction. This process repeated throughout training until the model completed its training or Early Stopping detected that validation performance was no longer improving.

V. CONCLUSION

The experiments showed that good neural network performance depends on finding the right balance between the training settings, model capacity, and available data. The initial experiments showed both underfitting and overfitting, while expanding the dataset from 30 to 900 balanced samples produced the largest improvement. Starting with EXP-05A, the model achieved around 99% validation accuracy and maintained strong precision, recall, and F1-score results. The later experiments also showed that increasing model capacity or changing more settings does not always lead to better performance. Overall, the exercise showed the importance of using enough representative data and monitoring both training and validation results to develop a model that can generalize well.